



Faculty of Computing

Department of Computing

Laboratory Manual

CS-250 : Data Structure and Algorithms

Class: BESE-16AB • Semester: Fall 2026

Experiment No. 03

Implementation of Linked List with its Operations

Date	25 September 2026
Time	09:00 AM–11:50 AM & 2:30 PM–5:00 PM
Instructor(s)	Dr Faisal Shafait
Lab Engineer	Hamza Fayyaz
Mapped CLO	CLO2 : Solve a given real time problem by applying appropriate data structure and algorithm. CLO3 : Practice programs using the latest IDEs ensuring testing, documentation and packaging of programs as per standards practices applicable to the software industry. CLO4 : Demonstrate commitment to professional ethics by following engineering norms applicable to the software industry.:

1 Introduction

Students have learned the fundamental concepts of linked lists in the lectures. This lab will introduce students with the practical implementation of a linked list and different operations that can be performed on a linked list.

2 Objectives

Objective of this lab is to get familiar with singly linked list and implement them in C++.

3 Tools and Software Requirements

Software: Visual Studio C++, VS Code

Helping Material: Lecture slides, Text book

4 Description

4.1 Singly Linked List

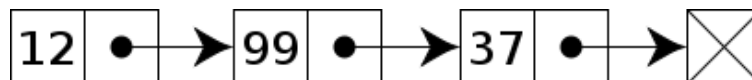
A Linked List is a data structure consisting of a group of nodes which together represent a sequence. Under the simplest form, each node is composed of two parts i.e. data part and a reference part (also known as, a link) to the next node in the sequence.

4.2 Linked List Structure

A linked list is made up of nodes, where each node contains:

- **Data:** The value the node stores.
- **Pointer:** A reference (pointer) to the next node in the list.

This structure allows efficient insertion or removal of elements from any position in the sequence.



4.3 Basic Operations

Creating the list

A linked list is created using nodes.

```
struct Node {  
    int data;        // Data stored in the node  
    Node* next;     // Pointer to the next node  
};
```

Initialize pointers to NULL

When working with linked lists, pointers must be initialized to NULL when they aren't pointing to any node. To create a linked list, initialize the head (starting node) to NULL to signify that the list is initially empty.

```
Node* head = NULL; // Head pointer initialized to NULL  
Node* temp = NULL; // Temporary pointer for list traversal or manipulation
```

Inserting nodes at beginning, last and from a specific location

At the beginning:

```
void insertAtBeginning(Node*& head, int value)
{
    // Dynamically allocate memory for a new node
    Node* newNode = new Node();
    // Assign the value to the node
    newNode->data = value;
    // Make the new node point to the current head
    newNode->next = head;
    // Update the head to the new node
    head = newNode;
}
```

At the end:

```
void insertAtEnd(Node*& head, int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;    // The new node will point to NULL (end of list)

    if (head == NULL) {    // If the list is empty, the new node becomes the head
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != NULL) { // Traverse to the last node
            temp = temp->next;
        }
        temp->next = newNode; // Link the last node to the new node
    }
}
```

At the specific position:

```
void insertAtPosition(Node*& head, int value, int position) {
    Node* newNode = new Node();
    newNode->data = value;

    if (position == 1) {
        newNode->next = head;
        head = newNode;
        return;
    }

    Node* temp = head;
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next; // Traverse to the position before the insertion point
    }

    if (temp == NULL) {
        cout << "Position out of bounds" << endl;
    } else {
        newNode->next = temp->next; // Link the new node to the next node
        temp->next = newNode;       // Link the previous node to the new node
    }
}
```

Deletion of nodes from beginning, last and from a specific location

Delete from Beginning

```

void deleteFromBeginning(Node*& head) {
    if (head == NULL) {
        cout << "List is empty" << endl;
        return;
    }
    Node* temp = head;        // Store the current head
    head = head->next;        // Update head to point to the next node
    delete temp;             // Free the memory of the old head
}

```

Delete from End

```

void deleteFromEnd(Node*& head) {
    if (head == NULL) {
        cout << "List is empty" << endl;
        return;
    }
    if (head->next == NULL) { // If there's only one node
        delete head;
        head = NULL;
        return;
    }
    Node* temp = head;
    while (temp->next->next != NULL) { // Traverse to the second last node
        temp = temp->next;
    }
    delete temp->next; // Delete the last node
    temp->next = NULL; // Update the second last node to point to NULL
}

```

Delete from a Specific Position

```

void deleteFromPosition(Node*& head, int position) {
    if (head == NULL) {
        cout << "List is empty" << endl;
        return;
    }
    if (position == 1) {
        Node* temp = head;
        head = head->next;
        delete temp;
        return;
    }
    Node* temp = head;
    for (int i = 1; i < position - 1 && temp != NULL; i++) {
        temp = temp->next; // Traverse to the node before the one to be deleted
    }
    if (temp == NULL || temp->next == NULL) {
        cout << "Position out of bounds" << endl;
        return;
    }
    Node* nodeToDelete = temp->next; // Store the node to be deleted
    temp->next = temp->next->next;    // Update the link to skip the node
    delete nodeToDelete;             // Free the memory of the deleted node
}

```

Traversing the list

To display or process all elements in the list, we need to traverse it from the head to the end.

```

void displayList(Node* head) {
    if (head == NULL) {
        cout << "List is empty" << endl;
        return;
    }

    Node* temp = head;
    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next; // Move to the next node
    }
    cout << "NULL" << endl; // Indicates the end of the list
}

```

Destroying the list

To completely free the memory used by the list, we can traverse it and delete each node.

```

void destroyList(Node*& head) {
    Node* temp;
    while (head != NULL) {
        temp = head;
        head = head->next; // Move to the next node
        delete temp;       // Free the memory of the current node
    }
    cout << "List destroyed" << endl;
}

```

5 Lab Tasks

Write a C++ program that can:

1. Create a simple linked list using functions to insert nodes at the head.
2. Make a function that can insert another node at 3rd location.
3. Make a function that can display the lists made in 1 and 2.
4. Write a function that can delete node from the linked list selected by the user. Display it as well.
5. Write a function that can count the number of nodes present in list.
6. Create menu in main function to give call to all of the above functions depending upon user's input.
7. Implement a queue using a singly linked list in C++. The queue should support the following operations:
 - enqueue(int value): Insert an element at the end of the queue.
 - dequeue(): Remove an element from the front of the queue.
 - peek(): Return the value of the front element without removing it.
 - isEmpty(): Check if the queue is empty.
 - display(): Print the queue elements.
8. Implement a circular linked list in C++ and perform the following operations:
 - Insert an element at the end.
 - Delete an element from the front.
 - Display all elements.

- Detect if the list is circular by checking if the last node points back to the first node.

Important Note: Please note that you have to develop your solution in C++ (OOP) i.e. using classes and objects. Solution written in a procedural style will not be accepted.

Hint: First you will create the relevant classes, and the functions will belong to the List class.

Listing 1. Class of node and required list functions

```
1 class Node {
2 public:
3     int value;
4     node *next;
5 };
6
7 void insertAtHead(int new_value);
8 void insertAtLocation(int location, int new_value);
9 void delete(int del_value);
10 void displayList();
11 void countList();
```

6 Deliverables

Compile a single Word document by filling in the solution part and submit this Word file on LMS. Insert the solution/answer in this document. You must show the implementation of the tasks in the designing tool, along with your complete Word document to get your work graded. You must also submit this Word document on the LMS.

7 Lab Rubrics

Assessment	Doesn't meet Expectation (1–2)	Meets Expectation (3–4)	Exceeds Expectation (5)	Marks
Software Problem Realization (CLO2)	The Student is unable to understand and outline the problem and doesn't use the relevant method to solve it.	The student requires some guidance to completely comprehend the problem and to differentiate the data structure and algorithm comprehensively.	The student fully understands the given problem, is able to analyze the relevant method to solve it, and develops a detailed program flow.	
Software Tool Usage (CLO3)	The student has no idea on how to use the basic tools of the software. The codes have syntax errors, and parts of the codes are missing. Also, they are unable to imitate the required output.	The student has a limited command on the basic tools of the software and operated it under supervision. The codes are correct in terms of their syntax, however, the program output is not always correct.	The student has full command on various tools available in the software. Furthermore, his/her coding is complete and functional, and the program output is correct. Moreover, they can easily manipulate the code to design a particular solution.	
Ethics and Adherence to Laboratory Safety Rules (CLO4)	The student does not behave according to the professional ethics by following ethical norms applicable to the software industry such as acknowledgement while using publicly available data/code. Disturbs the lab environment, doesn't take care of safety measures, and/or isn't punctual.	The student partially demonstrate their commitment to professional ethics by following ethical norms applicable to the software industry such as referencing and acknowledgement while using publicly available data/code. Exhibits better behavior, works by taking into account the safety measures, and is punctual.	The student clearly express the commitment to professional ethics by following ethical norms applicable to the software industry such as referencing and acknowledgement while using publicly available data/code. Encourages others to maintain lab decorum, and alerts them to follow safety measures.	